DESCRIPTION OF COURSEWORK

Course Code	COM102
Course Name	Programming Language C
Lecturer	Najla'a Ateeq Mohammed Draib
Academic Session	2025/09
Assessment Title	COM102 – Assignment (20%)

A. Introduction/ Situation/ Background Information

This assignment evaluates students' ability to effectively utilize the C language by describing syntax, analyzing and debugging programming code, and designing solutions for practical problems.

B. Course Learning Outcomes (CLO) covered

At the end of this assessment, students are able to:

CLO 1 Describe the C-language syntax.
CLO 2 Analyze and debug programming code.
CLO 3 Design codes to solve practical problems using C language.

C. University Policy on Academic Misconduct

1. Academic misconduct is a serious offense in Xiamen University Malaysia. It can be defined as any of the following:
   i. **Plagiarism** is submitting or presenting someone else's work, words, ideas, data or information as your own intentionally or unintentionally. This includes incorporating published and unpublished material, whether in manuscript, printed or electronic form into your work without acknowledging the source (the person and the work).
   ii. **Collusion** is two or more people collaborating on a piece of work (in part or whole) which is intended to be wholly individual and passed it off as own individual work.
   iii. **Cheating** is an act of dishonesty or fraud in order to gain an unfair advantage in an assessment. This includes using or attempting to use, or assisting another to use materials that are prohibited or inappropriate, commissioning work from a third party, falsifying data, or breaching any examination rules.

2. All assessments submitted must be the student's own work, without any materials generated by AI tools, including direct copying and pasting of text or paraphrasing. Any form of academic misconduct, including using prohibited materials or inappropriate assistance, is a serious offense and will result in a zero mark for the entire assessment or part of it. If there is more than one guilty party, such as in case of collusion, all parties involved will receive the same penalty.

D. Instruction to Students

This assignment is an individual task.

Submission Requirements

- ❖ Each student should submit the following items:
  - A **PDF** file uploaded on Moodle.
    - Please use the format: **'YourID_YourName_COM102_Assignment_202509.pdf'**.
    - The PDF report should contain all the source code, running results, and any additional information required. Attach the cover page and marking rubrics with your assignment in the PDF file.
  - A **C** file named **'YourID_YourName_COM102_Assignment_202509.c'**.
    - Do not convert your C source file to another file format. Penalty will be imposed (minus 5%) if you do so.
- ❖ Submit a soft copy of your report content (the PDF file) to Turnitin for similarity checking. The Turnitin similarity index should be below 30%.
  - **Turnitin Class ID: 50804621**
  - **Enrollment Key: 1@COM102**

  All of these documents should be compressed into a single .zip file for submission.

Deadline

- All submissions must be made by **6 PM on December 26, 2025 (Friday, Week 13)**.

Late Submission Policy

- Late submissions will incur a penalty of **5%** (out of 20%) from the total marks awarded.
- Submissions made **after 12 PM on December 27, 2025** will not be accepted unless there is a **strong justification** provided.

E. Evaluation Breakdown

No.	Component Title	Percentage (20%)
1.	Overall Functionality	20
2.	Error Handling & Validation	20
3.	Comments and Documentation	10
4.	File Handling	15
5.	Creativity & Extension	10
6.	Report Quality (Write-up)	10
7.	Flowchart	10
8.	Testing Results Section	5
	TOTAL	**100 %**

F. Task (s)

Project: Campus Maintenance Management System
In this assignment, you will design, implement, test, and document a single C program that manages campus maintenance requests. The system must use structures, user-defined functions, a menu-driven interface, input validation, and basic file handling to support the following operations:

- Log new maintenance requests
- Assign a technician to a request
- Update the status of a request
- Search technician
- Generate summary reports

You will submit **one complete task**, which includes:

- A working C program for the **Campus Maintenance Management System**
- A **flowchart** for the "Find Technician by ID" function
- A short **report** describing your program design, main features, and testing

1. Data Structures

1.1 Request Structure

Create a structure named `Request` to store information about each maintenance request. It should contain at least:

- **Request ID** (`int reqId`): unique number for each request (no duplicates).
- **Location** (`char location[50]`): where the issue is (e.g., `"A1-203"`).
- **Category** (`char category[20]`): type of problem (e.g., `"Plumbing"`, `"Electrical"`, `"AC"`).
- **Priority** (`int priority`): urgency from **1 (highest priority)** to **5 (lowest priority)**.
- **Status** (`char status[12]`): current status (e.g., `"NEW"`, `"ASSIGNED"`, `"DONE"`).
- **Technician ID** (`int techId`): ID of the assigned technician; use **-1** if no technician is assigned yet.

1.2 Technician Structure

Create a structure named `Technician` to store information about each technician. It should contain at least:

- **Technician ID** (`int techId`): unique ID for each technician.
- **Name** (`char name[60]`): technician's full name.
- **Specialty** (`char specialty[20]`): main skill (e.g., `"Electrical"`, `"Plumbing"`).

You may add more fields if you think they are useful (e.g., phone number).

2. Core Features (User-Defined Functions)

Implement **at least four** well-designed functions for the main actions.

2.1 Log a New Request

- Input all request details (ID, location, category, priority, etc.).
- Ensure `reqId` is unique.
- Validate `priority` (1–5) and non-empty text fields.
- Set `status = "NEW"` and `techId = -1` if not assigned.
- On invalid input, show a clear error and re-prompt.

2.2 List Requests

- Provide options to:
  - List **all** requests
  - List by **status** (e.g. only `"NEW"`)
  - List by **category** (e.g. only `"Plumbing"`)
- Display at least: `reqId`, `location`, `category`, `priority`, `status`, `techId`.
- If no match is found, print a message.

2.3 Assign Technician

- Ask for `reqId` and `techId`.
- Check that the request exists and the technician exists.
- If both exist:
  - Set `request.techId` to the technician's ID.
  - Change `status` to `"ASSIGNED"`.
- If not found, show an appropriate message.
- *(Optional)* Check that the technician's `specialty` matches the request `category`.

Initialize a small array of `Technician` records in your code (i.e. hard-code a few sample technicians), or add a program option to input/add new technicians. This ensures the system has technician data available for assignment and search.

2.4 Update Status

- Ask for `reqId` and locate the request.
- Allow only these transitions:
  - `"NEW"` → `"ASSIGNED"`
  - `"ASSIGNED"` → `"DONE"`
- For any other attempted change (e.g. `"NEW"` → `"DONE"`, or changing `"DONE"` back), print a message such as:

Invalid status transition. You must ASSIGN a request before marking it DONE.

2.5 Search Technician

- The program should prompt for a technician ID, use the `findTechnicianById` function to locate the technician in the array, and then display the technician's details if found (or an appropriate "not found" message if not).

2.6 Export NEW Requests to a File (with File Handling)

At the end of the program, add file handling that the summary report is written to a file:

- Open a file (for example, `new_requests.txt`) for **writing**.
- Go through the **in-memory** array of requests.
- Write **only the requests whose status is `"NEW"`** to this file, using your chosen text format (e.g., one request per line).
- Close the file.
- Display a message showing how many `"NEW"` requests were written.

3. Search & Validation Helpers

3.1 Search function – `findRequestById(reqId)`

- Search the `Request` array for the given `reqId`.
- Return the **index** of the matching request if found.
- Return **-1** if no request with that ID exists.

3.2 Search function – findTechnicianById(techId)

- Search the `Technician` array for the given `techId`.
- Return the **index** of the matching technician if found.
- Return **-1** if no technician with that ID exists.

3.3 Validation Rules

Throughout the program:

- Reject duplicate `reqId` and `techId`.
- Check numeric ranges (e.g. priority 1–5).
- Enforce valid status transitions.
- Display **meaningful** error messages (not just `"Error"`).

4. Menu-Driven Interface (main)

Provide a looped menu until the user chooses to exit. Suggested menu:

- Log new request
- List requests (all / by status / by category)
- Assign technician
- Update status
- Search technician (for finding a technician by ID)
- Summary report (export NEW requests to file)
- Exit

5. Flowchart – Find Technician by ID

Draw a flowchart for a function that searches the technician array by technician ID and returns the position if found, or -1 if not found.

6. Testing and Results Summary (CLO2)

In your report, include a short section listing the test cases you tried (e.g., valid/invalid input, status transition checks, edge cases) and the outcomes. Describe any issues you encountered and how you fixed them. This demonstrates your ability to test and debug the program effectively.

APPENDIX 1

MARKING RUBRICS

Component Title	COM102 - Assignment (20%)				Percentage (%)	100%	
Criteria	Score and Descriptors					Weight (%)	Marks
	Excellent (17-20)	Good (13-16)	Average (9-12)	Need Improvement (5-8)	Poor (1-4)		
Overall Functionality (CLO3)	All required features work correctly; menu fully functional; all operations behave as specified.	Most core features work; only minor bugs or missing small options.	Several main features work, but some options are missing or clearly incorrect.	Only a few features partially work; many menu options are broken or wrong.	Program does not compile/run, or almost all required features are missing.	**20%**	
	(13-15)	**(10-12)**	**(7-9)**	**(4-6)**	**(1-3)**		
Error Handling & Debugging (CLO2)	Thorough input checks and debugging. Program safely handles invalid input and behaves as expected in all test cases.	Minor gaps in validation or error handling; mostly reliable behavior.	Some validation present; debugging incomplete or inconsistent.	Limited validation; program often misbehaves or crashes on bad input.	No meaningful validation or debugging; frequent crashes or incorrect outputs	**20%**	
Comments and Documentation (CLO1)	Clear, meaningful comments for functions, key logic, and variables; easy to read; header info included.	Helpful comments in most places; code is understandable with small gaps.	Few comments; only obvious parts documented; reading the whole code is needed to understand.	Minimal comments; program logic is hard to follow.	No useful comments or documentation.	**10%**	
File Handling (CLO3)	File handling is implemented correctly: exit saves all requests, and the summary report exports "NEW" requests with proper error handling.	File handling mostly correct; minor issues but files remain usable.	Read/write implemented but with noticeable mistakes.	File handling is unreliable; save or export often fails or does not follow the specification.	No file handling, or implementation does not match the requirements at all.	**15%**	
Creativity & Extension (CLO3)	Adds meaningful, well-implemented enhancements beyond the minimum	Some useful enhancements beyond the basic requirements.	Small improvements or minor extras; mostly just the base features.	Very limited extension; almost exactly the minimum specification.	No extension; clearly below what is reasonably achievable.	**10%**	
	(9-10)	**(7-8)**	**(5-6)**	**(3-4)**	**(1-2)**		
Report Quality (Write-up) (CLO1)	The written report is well-organized, clearly written, and consistent with the program's implementation.	Clear and mostly well-organized; minor missing parts or small inconsistencies.	Covers main points but brief, weakly structured, or has several unclear sections.	Poorly organized or very short; key explanations are missing or unclear.	No report, or report is unrelated or not understandable.	**10%**	
Flowchart (CLO3)	Flowchart accurately represents the search logic with correct symbols and structure.	Flowchart mostly clear and correct; minor omissions or notation issues.	General idea is visible, but some steps or decisions are missing or confusing.	Flowchart is unclear, incomplete, or only loosely related to the required search.	No flowchart submitted, or it does not represent the task.	**10%**	
Testing Results (CLO2)	The report includes a clear testing section with varied test cases and thoughtful explanation of program behavior and debugging steps.	Several test cases are shown, including some edge or error cases. Student mentions a few corrections or testing outcomes.	Basic test cases provided but lacks depth (e.g., only valid inputs, no debugging discussion).	Few or unclear test cases. Minimal insight into how the program was tested	No meaningful test results shown. No sign of debugging or analysis effort.	**5%**	
TOTAL						**100%**	

Note to students: Please include the marking rubric when submitting your coursework.